Apex Institute of Technology

Computer Science & Engineering

Experiment 9

Name: Shivam Abhepal **UID: 24BAI70214**

Branch: B.E. CSE (AIML) **Section: 24AIT_KRG-G1**

Semester: 4 **Date of Performance: 10.04.2026**

Subject Name: Database Management System **Subject Code: 24CSH-298**

AIM: To understand and implement database triggers in PostgreSQL to automate data validation and computational logic, ensuring data integrity by enforcing business rules during DML operations.

OBJECTIVES:

- To understand the concept of database triggers
- To implement BEFORE INSERT trigger in PostgreSQL
- To automate calculation of total payable amount
- To enforce validation rules using triggers
- To handle exceptions using PL/pgSQL

SOFTWARE REQUIREMENTS:

- PostgreSQL (pgAdmin / DB Fiddle)
- SQL (DDL, DML)
- PL/pgSQL

PROCEDURE/STEPS:

1. Start the system and log in
2. Open pgAdmin / DB Fiddle
3. Connect to the database

4. Open Query Tool
5. Create the employee table
6. Create trigger function
7. Create trigger on table
8. Insert records into table
9. Observe trigger execution and output

PRACTICAL/EXPERIMENTAL STEPS:

```
-----------------------------------------------
-- Create Table
-----------------------------------------------
DROP TABLE IF EXISTS employees;
CREATE TABLE employee (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   working_hours INT,
   perhour_salary NUMERIC,
   total_payable_amount NUMERIC
);
-----------------------------------------------
-- Create Trigger Function
-----------------------------------------------
CREATE OR REPLACE FUNCTION CALCULATE_AMOUNT()
RETURNS TRIGGER
AS
$$
BEGIN
   NEW.total_payable_amount = NEW.perhour_salary * NEW.working_hours;

   IF NEW.total_payable_amount > 25000 THEN
      RAISE EXCEPTION 'AMOUNT IS GREATER THAN 25000';
   END IF;

   RETURN NEW;
END;
$$
```

```
LANGUAGE PLPGSQL;

-----------------------------------------------
-- Create Trigger
-----------------------------------------------

CREATE OR REPLACE TRIGGER CAL_PAYABLE_AMOUNT

BEFORE INSERT

ON EMPLOYEE

FOR EACH ROW

EXECUTE FUNCTION CALCULATE_AMOUNT();

-----------------------------------------------

-- Insert Valid Data

-----------------------------------------------

DO

$$

BEGIN

   INSERT INTO EMPLOYEE(emp_id, emp_name, working_hours, perhour_salary)

   VALUES (1, 'AKASH', 10, 250);


EXCEPTION

   WHEN OTHERS THEN
```

```
    RAISE NOTICE '%', SQLERRM;
END;
$$;
-----------------------------------------------
-- View Data
-----------------------------------------------
SELECT * FROM EMPLOYEE;
-----------------------------------------------
-- Insert Invalid Data
-----------------------------------------------
DO
$$
BEGIN
  INSERT INTO EMPLOYEE(emp_id, emp_name, working_hours, perhour_salary)
  VALUES (2, 'AKASH', 10, 250000);

EXCEPTION
  WHEN OTHERS THEN
    RAISE NOTICE '%', SQLERRM;
```

END;

$$;

OUTPUT:(Screenshots)

- **Valid Insert**

Query | Query History

```
1  DROP TABLE IF EXISTS employee;
2  CREATE TABLE employee (
3      emp_id INT PRIMARY KEY,
4      emp_name VARCHAR(50),
5      working_hours INT,
6      perhour_salary NUMERIC,
7      total_payable_amount NUMERIC
8  );
9
10 --CREATING TRIGGER FUNCTION
11 CREATE OR REPLACE FUNCTION CALCULATE_AMOUNT()
12 RETURNS TRIGGER
13 AS
14 $$
15 BEGIN
16     NEW.total_payable_amount=NEW.perhour_salary*NEW.working_hours;
17     IF NEW.total_payable_amount>25000 THEN
18         RAISE EXCEPTION 'AMOUNT IS GREATER THAN 25000';
19     END IF;
20     RETURN NEW;
21 END;
22 $$
```

Data Output | Messages | Notifications

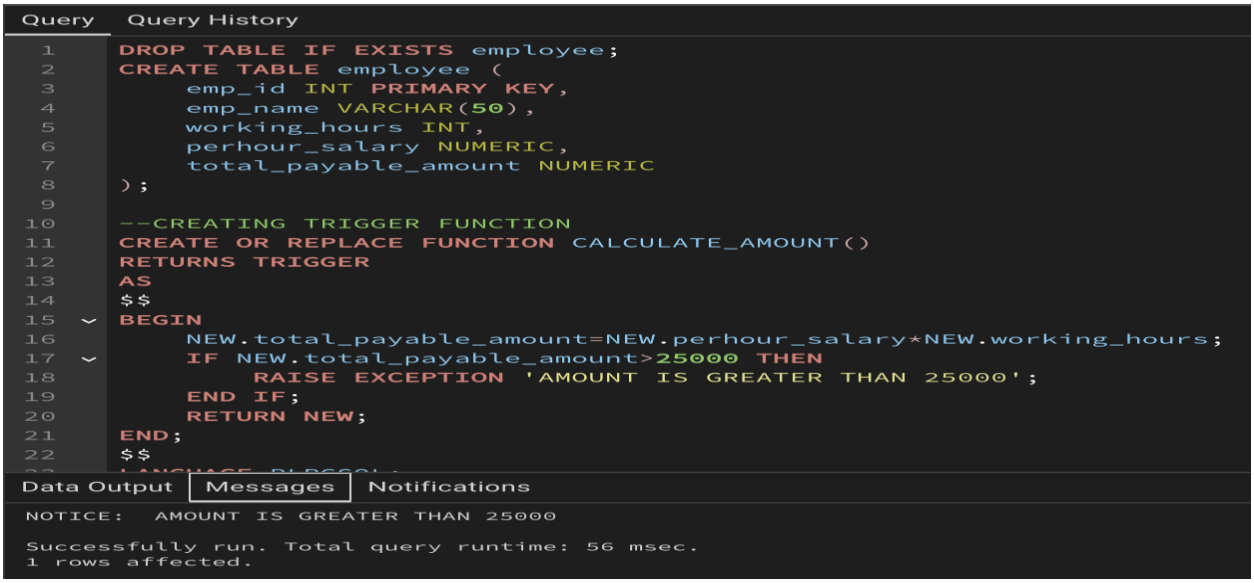
SQL

	emp_id [PK] integer	emp_name character varying (50)	working_hours integer	perhour_salary numeric	total_payable_amount numeric
1	1	Jaskaran	8	250	2000

Apex Institute of Technology

Computer Science & Engineering

- **Invalid Insert**



I/O ANALYSIS:

Input:

- Employee ID
- Employee Name
- Working Hours
- Per Hour Salary

Output:

- Automatically calculated total payable amount
- Error message if payable exceeds limit

LEARNING OUTCOMES:

- Understood database triggers in PostgreSQL
- Learned how to automate calculations using triggers
- Implemented data validation using business rules
- Gained knowledge of BEFORE INSERT triggers

- Handled exceptions using PL/pgSQL

RESULT:

The trigger successfully calculates the total payable amount and restricts insertion of records where the amount exceeds the defined limit, thereby maintaining data integrity.